Principal Component Analysis (PCA) - Comprehensive Tutorial

Instructor: Dr. Arun B Ayyar
Institution: IIT Madras
Course: DA5400W - Foundations of Machine Learning

Table of Contents

1. Introduction to Dimensionality Reduction
2. Mathematical Foundation of PCA
3. Step-by-Step PCA Algorithm
4. Geometric Interpretation
5. PCA from Scratch Implementation
6. PCA with Scikit-learn
7. Choosing the Number of Components
8. Applications and Use Cases
9. Limitations and Considerations
10. Practice Exercises

In [ ]:
```python
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris, load_digits, make_classification
import pandas as pd

# Set style for better visualizations
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
np.random.seed(42)

print("Libraries imported successfully!")
```

Part 1: Introduction to Dimensionality Reduction

What is Dimensionality Reduction?

Dimensionality reduction is the process of reducing the number of features (dimensions) in a dataset while preserving as much information as possible.

Why Do We Need It?

1. **Curse of Dimensionality**: As the number of features increases, the amount of data needed to maintain statistical significance grows exponentially
2. **Visualization**: Humans can only visualize 2D or 3D data effectively
3. **Computational Efficiency**: Fewer dimensions mean faster training and prediction
4. **Noise Reduction**: Removing less important features can improve model performance
5. **Feature Extraction**: Discover underlying patterns in high-dimensional data

What is PCA?

Principal Component Analysis (PCA) is an unsupervised linear dimensionality reduction technique that:

- Finds new axes (principal components) that maximize variance
- Projects data onto these new axes
- Reduces dimensionality by keeping only the top k components

Key Properties:

- Principal components are **orthogonal** (perpendicular) to each other
- First component captures the **most variance**, second captures the second most, etc.
- Components are **linear combinations** of original features

In [ ]:
```
# Example: High-dimensional data problem
# Create a dataset with 100 features
X_high_dim, y = make_classification(n_samples=1000, n_features=100,
                                    n_informative=10, n_redundant=80,
                                    n_repeated=10, random_state=42)

print(f"Original data shape: {X_high_dim.shape}")
print(f"Number of samples: {X_high_dim.shape[0]}")
print(f"Number of features: {X_high_dim.shape[1]}")
print(f"\nChallenge: How do we visualize or understand 100-dimensional data?")
print("Solution: Use PCA to reduce to 2 or 3 dimensions!")
```

Part 2: Mathematical Foundation of PCA

Variance and Covariance

Variance measures the spread of a single variable:

$$\text{Var}(X) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Covariance measures how two variables vary together:

$$\text{Cov}(X, Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

Covariance Matrix for d features:

$$\Sigma = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_d) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots & \text{Cov}(X_2, X_d) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_d, X_1) & \text{Cov}(X_d, X_2) & \cdots & \text{Var}(X_d) \end{bmatrix}$$

Eigenvalues and Eigenvectors

For a square matrix A and vector v:

$$Av = \lambda v$$

Where:

- v is an **eigenvector** (direction that doesn't change under transformation)
- λ is an **eigenvalue** (scaling factor in that direction)

Key Insight: The eigenvectors of the covariance matrix are the principal components, and the eigenvalues represent the variance explained by each component.

In [ ]:
```
# Demonstration: Covariance and correlation
# Create correlated 2D data
np.random.seed(42)
mean = [0, 0]
cov = [[3, 2.5],    # Covariance matrix: variance of X=3, covariance=2.5
       [2.5, 3]]    # variance of Y=3
data_2d = np.random.multivariate_normal(mean, cov, 500)

# Calculate covariance matrix
cov_matrix = np.cov(data_2d.T)

print("Covariance Matrix:")
print(cov_matrix)
print(f"\nVariance of X: {cov_matrix[0, 0]:.2f}")
print(f"Variance of Y: {cov_matrix[1, 1]:.2f}")
print(f"Covariance(X, Y): {cov_matrix[0, 1]:.2f}")
print(f"\nPositive covariance means X and Y tend to increase together")

# Visualize
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Scatter plot
```

```
axes[0].scatter(data_2d[:, 0], data_2d[:, 1], alpha=0.6, s=30)
axes[0].axhline(y=0, color='k', linestyle='--', linewidth=0.5)
axes[0].axvline(x=0, color='k', linestyle='--', linewidth=0.5)
axes[0].set_xlabel('X', fontsize=12)
axes[0].set_ylabel('Y', fontsize=12)
axes[0].set_title('2D Data with Positive Covariance', fontsize=14)
axes[0].grid(True, alpha=0.3)

# Covariance matrix heatmap
sns.heatmap(cov_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            xticklabels=['X', 'Y'], yticklabels=['X', 'Y'], ax=axes[1],
            cbar_kws={'label': 'Covariance'})
axes[1].set_title('Covariance Matrix', fontsize=14)

plt.tight_layout()
plt.show()
```

In [ ]:

```
# Demonstration: Eigenvectors and eigenvalues
# Compute eigenvectors and eigenvalues of the covariance matrix
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

print("Eigenvalues (variance explained by each component):")
for i, eigenval in enumerate(eigenvalues):
    print(f"  PC{i+1}: {eigenval:.3f}")

print("\nEigenvectors (principal component directions):")
for i, eigenvec in enumerate(eigenvectors.T):
    print(f"  PC{i+1}: [{eigenvec[0]:.3f}, {eigenvec[1]:.3f}]")

# Visualize eigenvectors on the data
plt.figure(figsize=(10, 8))
plt.scatter(data_2d[:, 0], data_2d[:, 1], alpha=0.5, s=30, label='Data')

# Plot eigenvectors scaled by eigenvalues
origin = np.mean(data_2d, axis=0)
for i, (eigenval, eigenvec) in enumerate(zip(eigenvalues, eigenvectors.T)):
    # Scale eigenvector by square root of eigenvalue (standard deviation)
    plt.arrow(origin[0], origin[1],
              eigenvec[0] * np.sqrt(eigenval) * 2,
              eigenvec[1] * np.sqrt(eigenval) * 2,
              head_width=0.3, head_length=0.3, fc=f'C{i+1}', ec=f'C{i+1}',
              linewidth=3, label=f'PC{i+1} (λ={eigenval:.2f})')

plt.axhline(y=0, color='k', linestyle='--', linewidth=0.5)
plt.axvline(x=0, color='k', linestyle='--', linewidth=0.5)
plt.xlabel('X', fontsize=12)
plt.ylabel('Y', fontsize=12)
plt.title('Principal Components as Eigenvectors\n(Arrow length = 2×√eigenvalue)', f
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.tight_layout()
plt.show()

print("\nKey Observations:")
print("1. PC1 (red) points in the direction of maximum variance")
```

```
print("2. PC2 (orange) is perpendicular to PC1")
print("3. Eigenvalue of PC1 > Eigenvalue of PC2 (captures more variance)")
```

Part 3: Step-by-Step PCA Algorithm

The PCA Algorithm

Input: Data matrix X of shape (n, d) where n = samples, d = features

Steps:

1. **Standardize the data** (mean = 0, std = 1 for each feature)

$$X_{\text{std}} = \frac{X - \mu}{\sigma}$$

2. **Compute the covariance matrix**

$$\Sigma = \frac{1}{n-1} X_{\text{std}}^T X_{\text{std}}$$

3. **Compute eigenvalues and eigenvectors** of Σ

$$\Sigma v_i = \lambda_i v_i$$

4. **Sort eigenvectors** by eigenvalues in descending order

5. **Select top k eigenvectors** to form projection matrix W

6. **Transform the data**

$$X_{\text{PCA}} = X_{\text{std}} \cdot W$$

Output: Reduced data matrix of shape (n, k)

Why Standardization?

Features with larger scales dominate the variance. Standardization ensures all features contribute equally.

In [ ]:
```
# Step-by-step PCA implementation
# Using the 2D data from before

print("=" * 60)
print("STEP-BY-STEP PCA IMPLEMENTATION")
print("=" * 60)

# Step 1: Standardize the data
print("\nStep 1: Standardize the data")
print("-" * 40)
```

```python
mean = np.mean(data_2d, axis=0)
std = np.std(data_2d, axis=0)
data_standardized = (data_2d - mean) / std

print(f"Original mean: {mean}")
print(f"Original std: {std}")
print(f"Standardized mean: {np.mean(data_standardized, axis=0)}")
print(f"Standardized std: {np.std(data_standardized, axis=0)}")

# Step 2: Compute covariance matrix
print("\nStep 2: Compute covariance matrix")
print("-" * 40)
cov_matrix = np.cov(data_standardized.T)
print("Covariance matrix:")
print(cov_matrix)

# Step 3: Compute eigenvalues and eigenvectors
print("\nStep 3: Compute eigenvalues and eigenvectors")
print("-" * 40)
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
print(f"Eigenvalues: {eigenvalues}")
print("Eigenvectors:")
print(eigenvectors)

# Step 4: Sort by eigenvalues
print("\nStep 4: Sort eigenvectors by eigenvalues (descending)")
print("-" * 40)
idx = eigenvalues.argsort()[::-1]  # Descending order
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
print(f"Sorted eigenvalues: {eigenvalues}")
print("Sorted eigenvectors:")
print(eigenvectors)

# Step 5: Select top k components (k=1 for 2D to 1D reduction)
print("\nStep 5: Select top k=1 component")
print("-" * 40)
k = 1
W = eigenvectors[:, :k]  # Projection matrix
print(f"Projection matrix W (shape {W.shape}):")
print(W)

# Step 6: Transform the data
print("\nStep 6: Transform the data")
print("-" * 40)
data_pca = data_standardized @ W
print(f"Original data shape: {data_2d.shape}")
print(f"Transformed data shape: {data_pca.shape}")
print(f"\nFirst 5 transformed samples:")
print(data_pca[:5])

# Calculate variance explained
variance_explained = eigenvalues / np.sum(eigenvalues) * 100
print(f"\nVariance explained by PC1: {variance_explained[0]:.2f}%")
print(f"Variance explained by PC2: {variance_explained[1]:.2f}%")
```

Part 4: Geometric Interpretation

What Does PCA Do Geometrically?

1. **Rotation**: PCA rotates the coordinate system to align with the directions of maximum variance
2. **Projection**: Data is projected onto the new axes (principal components)
3. **Dimensionality Reduction**: Keep only the top k components, discarding the rest

Intuition

Imagine you have a 3D cloud of points shaped like a flat pancake. PCA finds:

- **PC1**: The longest axis of the pancake
- **PC2**: The second longest axis (perpendicular to PC1)
- **PC3**: The thickness of the pancake (very small)

By keeping only PC1 and PC2, you capture most of the information while reducing from 3D to 2D.

In [ ]:
```
# Visualization: Before and after PCA
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Original 2D data
axes[0].scatter(data_2d[:, 0], data_2d[:, 1], alpha=0.6, s=30)
axes[0].axhline(y=0, color='k', linestyle='--', linewidth=0.5)
axes[0].axvline(x=0, color='k', linestyle='--', linewidth=0.5)
axes[0].set_xlabel('Original X', fontsize=12)
axes[0].set_ylabel('Original Y', fontsize=12)
axes[0].set_title('Original 2D Data', fontsize=14)
axes[0].grid(True, alpha=0.3)
axes[0].axis('equal')

# Data with principal components overlaid
axes[1].scatter(data_standardized[:, 0], data_standardized[:, 1], alpha=0.6, s=30)
origin = [0, 0]
for i, (eigenval, eigenvec) in enumerate(zip(eigenvalues, eigenvectors.T)):
    axes[1].arrow(origin[0], origin[1],
                  eigenvec[0] * np.sqrt(eigenval) * 2,
                  eigenvec[1] * np.sqrt(eigenval) * 2,
                  head_width=0.15, head_length=0.15, fc=f'C{i+1}', ec=f'C{i+1}',
                  linewidth=3, label=f'PC{i+1}')
axes[1].axhline(y=0, color='k', linestyle='--', linewidth=0.5)
axes[1].axvline(x=0, color='k', linestyle='--', linewidth=0.5)
axes[1].set_xlabel('Standardized X', fontsize=12)
axes[1].set_ylabel('Standardized Y', fontsize=12)
axes[1].set_title('Standardized Data + Principal Components', fontsize=14)
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)
axes[1].axis('equal')
```

```
# Projected onto PC1 (1D)
axes[2].scatter(data_pca, np.zeros_like(data_pca), alpha=0.6, s=30)
axes[2].axhline(y=0, color='k', linewidth=1)
axes[2].set_xlabel('PC1', fontsize=12)
axes[2].set_ylabel('(Reduced to 1D)', fontsize=12)
axes[2].set_title(f'After PCA: 2D → 1D\n({variance_explained[0]:.1f}% variance reta
axes[2].set_ylim(-0.5, 0.5)
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nGeometric Interpretation:")
print("1. Left: Original data with correlation between X and Y")
print("2. Middle: Principal components (red and orange arrows) show new axes")
print("3. Right: Data projected onto PC1 only (dimensionality reduced from 2D to 1D
```

Part 5: PCA from Scratch - Complete Implementation

Let's implement a complete PCA class from scratch to solidify our understanding.

In [ ]:
```
class PCA_FromScratch:
    """
    Principal Component Analysis implementation from scratch.
    """

    def __init__(self, n_components=2):
        """
        Initialize PCA.

        Parameters:
        -----------
        n_components : int
            Number of principal components to keep
        """
        self.n_components = n_components
        self.components = None
        self.mean = None
        self.std = None
        self.eigenvalues = None

    def fit(self, X):
        """
        Fit PCA on the data.

        Parameters:
        -----------
        X : array-like, shape (n_samples, n_features)
            Training data
        """
```

```python
        # Step 1: Standardize
        self.mean = np.mean(X, axis=0)
        self.std = np.std(X, axis=0)
        X_std = (X - self.mean) / self.std

        # Step 2: Covariance matrix
        cov_matrix = np.cov(X_std.T)

        # Step 3: Eigenvalues and eigenvectors
        eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

        # Step 4: Sort by eigenvalues
        idx = eigenvalues.argsort()[::-1]
        eigenvalues = eigenvalues[idx]
        eigenvectors = eigenvectors[:, idx]

        # Step 5: Select top k components
        self.components = eigenvectors[:, :self.n_components]
        self.eigenvalues = eigenvalues[:self.n_components]

        return self

    def transform(self, X):
        """
        Transform data to principal component space.

        Parameters:
        -----------
        X : array-like, shape (n_samples, n_features)
            Data to transform

        Returns:
        --------
        X_transformed : array-like, shape (n_samples, n_components)
            Transformed data
        """
        # Standardize using training statistics
        X_std = (X - self.mean) / self.std

        # Project onto principal components
        return X_std @ self.components

    def fit_transform(self, X):
        """
        Fit and transform in one step.
        """
        self.fit(X)
        return self.transform(X)

    def explained_variance_ratio(self):
        """
        Return the proportion of variance explained by each component.
        """
        total_variance = np.sum(self.eigenvalues)
        return self.eigenvalues / total_variance
```

```
    def inverse_transform(self, X_transformed):
        """
        Transform data back to original space.

        Parameters:
        -----------
        X_transformed : array-like, shape (n_samples, n_components)
            Transformed data

        Returns:
        --------
        X_reconstructed : array-like, shape (n_samples, n_features)
            Reconstructed data (approximation)
        """
        X_std = X_transformed @ self.components.T
        return X_std * self.std + self.mean

print("PCA_FromScratch class defined successfully!")
```

In [ ]:

```
# Test our implementation
print("Testing PCA_FromScratch implementation\n")
print("=" * 60)

# Create test data
np.random.seed(42)
X_test = np.random.randn(100, 5)  # 100 samples, 5 features

# Apply our PCA
pca_scratch = PCA_FromScratch(n_components=2)
X_transformed = pca_scratch.fit_transform(X_test)

print(f"Original shape: {X_test.shape}")
print(f"Transformed shape: {X_transformed.shape}")
print(f"\nExplained variance ratio: {pca_scratch.explained_variance_ratio()}")
print(f"Total variance explained: {np.sum(pca_scratch.explained_variance_ratio()):.

# Reconstruct data
X_reconstructed = pca_scratch.inverse_transform(X_transformed)
reconstruction_error = np.mean((X_test - X_reconstructed) ** 2)
print(f"\nReconstruction error (MSE): {reconstruction_error:.6f}")

print("\n" + "=" * 60)
print("✓ Implementation working correctly!")
```

Part 6: PCA with Scikit-learn

While understanding the mathematics is crucial, in practice we use optimized libraries like scikit-learn.

In [ ]:

```
# Load the famous Iris dataset
iris = load_iris()
X_iris = iris.data
```

```
y_iris = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

print("Iris Dataset")
print("=" * 60)
print(f"Number of samples: {X_iris.shape[0]}")
print(f"Number of features: {X_iris.shape[1]}")
print(f"Feature names: {feature_names}")
print(f"Target names: {target_names}")
print(f"\nFirst 5 samples:")
print(pd.DataFrame(X_iris[:5], columns=feature_names))
```

In [ ]:

```
# Apply PCA using scikit-learn
# Standardize first
scaler = StandardScaler()
X_iris_scaled = scaler.fit_transform(X_iris)

# Apply PCA to reduce from 4D to 2D
pca_sklearn = PCA(n_components=2)
X_iris_pca = pca_sklearn.fit_transform(X_iris_scaled)

print("PCA with Scikit-learn")
print("=" * 60)
print(f"Original shape: {X_iris.shape}")
print(f"Transformed shape: {X_iris_pca.shape}")
print(f"\nExplained variance ratio:")
for i, var in enumerate(pca_sklearn.explained_variance_ratio_):
    print(f"  PC{i+1}: {var:.4f} ({var*100:.2f}%)")
print(f"\nTotal variance explained: {np.sum(pca_sklearn.explained_variance_ratio_):

print(f"\nPrincipal components (loadings):")
components_df = pd.DataFrame(
    pca_sklearn.components_,
    columns=feature_names,
    index=['PC1', 'PC2']
)
print(components_df)
```

In [ ]:

```
# Visualize Iris dataset in 2D after PCA
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Scatter plot colored by species
colors = ['red', 'green', 'blue']
for i, (target, name) in enumerate(zip(range(3), target_names)):
    mask = y_iris == target
    axes[0].scatter(X_iris_pca[mask, 0], X_iris_pca[mask, 1],
                    c=colors[i], label=name, alpha=0.7, s=60, edgecolors='k', linew

axes[0].set_xlabel(f'PC1 ({pca_sklearn.explained_variance_ratio_[0]*100:.1f}% varia
axes[0].set_ylabel(f'PC2 ({pca_sklearn.explained_variance_ratio_[1]*100:.1f}% varia
axes[0].set_title('Iris Dataset: PCA Projection (4D → 2D)', fontsize=14, fontweight
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)
```

```
# Component loadings (biplot)
for i, (target, name) in enumerate(zip(range(3), target_names)):
    mask = y_iris == target
    axes[1].scatter(X_iris_pca[mask, 0], X_iris_pca[mask, 1],
                    c=colors[i], label=name, alpha=0.5, s=40)

# Plot feature vectors
for i, feature in enumerate(feature_names):
    axes[1].arrow(0, 0,
                  pca_sklearn.components_[0, i] * 3,
                  pca_sklearn.components_[1, i] * 3,
                  head_width=0.15, head_length=0.15, fc='black', ec='black', linewi
    axes[1].text(pca_sklearn.components_[0, i] * 3.3,
                 pca_sklearn.components_[1, i] * 3.3,
                 feature.replace(' (cm)', ''), fontsize=10, fontweight='bold')

axes[1].set_xlabel(f'PC1 ({pca_sklearn.explained_variance_ratio_[0]*100:.1f}% varia
axes[1].set_ylabel(f'PC2 ({pca_sklearn.explained_variance_ratio_[1]*100:.1f}% varia
axes[1].set_title('PCA Biplot: Data + Feature Loadings', fontsize=14, fontweight='b
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nInterpretation:")
print("- Left plot: Iris species are well-separated in the 2D PCA space")
print("- Right plot (biplot): Arrows show how original features contribute to PCs")
print("  * Longer arrow = stronger contribution")
print("  * Arrow direction = correlation with PCs")
```

Part 7: Choosing the Number of Components

Methods for Selecting k

1. **Explained Variance Threshold**: Keep components until you reach a target (e.g., 95% variance)
2. **Scree Plot**: Look for the "elbow" where eigenvalues drop off
3. **Cumulative Variance Plot**: Visualize cumulative variance explained
4. **Cross-Validation**: Use downstream task performance to select k

Rule of Thumb

- **Visualization**: k = 2 or 3
- **Dimensionality Reduction**: Keep 80-95% of variance
- **Noise Reduction**: Keep components with eigenvalues > 1 (Kaiser criterion)

In [ ]:
```
# Fit PCA with all components
pca_full = PCA()
```

```
pca_full.fit(X_iris_scaled)

# Create visualizations for choosing k
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 1. Scree plot
axes[0].bar(range(1, len(pca_full.explained_variance_) + 1),
            pca_full.explained_variance_, alpha=0.7, color='steelblue')
axes[0].plot(range(1, len(pca_full.explained_variance_) + 1),
             pca_full.explained_variance_, 'ro-', linewidth=2, markersize=8)
axes[0].set_xlabel('Principal Component', fontsize=12)
axes[0].set_ylabel('Eigenvalue (Variance)', fontsize=12)
axes[0].set_title('Scree Plot', fontsize=14, fontweight='bold')
axes[0].set_xticks(range(1, len(pca_full.explained_variance_) + 1))
axes[0].grid(True, alpha=0.3, axis='y')

# Add Kaiser criterion line (eigenvalue = 1)
axes[0].axhline(y=1, color='red', linestyle='--', linewidth=2, label='Kaiser criter
axes[0].legend(fontsize=10)

# 2. Explained variance ratio
axes[1].bar(range(1, len(pca_full.explained_variance_ratio_) + 1),
            pca_full.explained_variance_ratio_ * 100, alpha=0.7, color='coral')
axes[1].set_xlabel('Principal Component', fontsize=12)
axes[1].set_ylabel('Variance Explained (%)', fontsize=12)
axes[1].set_title('Variance Explained by Each PC', fontsize=14, fontweight='bold')
axes[1].set_xticks(range(1, len(pca_full.explained_variance_ratio_) + 1))
axes[1].grid(True, alpha=0.3, axis='y')

# 3. Cumulative variance explained
cumsum = np.cumsum(pca_full.explained_variance_ratio_) * 100
axes[2].plot(range(1, len(cumsum) + 1), cumsum, 'go-', linewidth=3, markersize=10)
axes[2].fill_between(range(1, len(cumsum) + 1), cumsum, alpha=0.3, color='green')
axes[2].axhline(y=95, color='red', linestyle='--', linewidth=2, label='95% threshol
axes[2].axhline(y=90, color='orange', linestyle='--', linewidth=2, label='90% thres
axes[2].set_xlabel('Number of Components', fontsize=12)
axes[2].set_ylabel('Cumulative Variance Explained (%)', fontsize=12)
axes[2].set_title('Cumulative Variance Explained', fontsize=14, fontweight='bold')
axes[2].set_xticks(range(1, len(cumsum) + 1))
axes[2].set_ylim([0, 105])
axes[2].legend(fontsize=10)
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Print detailed information
print("\nVariance Explained by Each Component:")
print("=" * 60)
for i, (var, cumvar) in enumerate(zip(pca_full.explained_variance_ratio_, cumsum)):
    print(f"PC{i+1}: {var*100:.2f}% (Cumulative: {cumvar:.2f}%)")

# Recommendation
n_components_90 = np.argmax(cumsum >= 90) + 1
n_components_95 = np.argmax(cumsum >= 95) + 1
```

```
print("\n" + "=" * 60)
print("Recommendations:")
print(f"  • For 90% variance: Use {n_components_90} components")
print(f"  • For 95% variance: Use {n_components_95} components")
print(f"  • For visualization: Use 2 components ({cumsum[1]:.1f}% variance)")
```

Part 8: Applications and Use Cases

Common Applications of PCA

1. **Visualization**: Reduce high-dimensional data to 2D/3D for plotting
2. **Noise Reduction**: Remove components with low variance (likely noise)
3. **Feature Extraction**: Create new features that capture most variance
4. **Data Compression**: Reduce storage requirements
5. **Preprocessing**: Improve machine learning model performance
6. **Exploratory Data Analysis**: Understand data structure and relationships

Example: Handwritten Digit Recognition

In [ ]:
```
# Load digits dataset (8x8 images = 64 features)
digits = load_digits()
X_digits = digits.data
y_digits = digits.target

print("Handwritten Digits Dataset")
print("=" * 60)
print(f"Number of samples: {X_digits.shape[0]}")
print(f"Number of features (pixels): {X_digits.shape[1]}")
print(f"Image shape: 8×8 pixels")
print(f"Number of classes: {len(np.unique(y_digits))}")

# Display some sample digits
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_digits[i].reshape(8, 8), cmap='gray')
    ax.set_title(f'Label: {y_digits[i]}', fontsize=12)
    ax.axis('off')
plt.suptitle('Sample Handwritten Digits (8×8 = 64 pixels)', fontsize=14, fontweight
plt.tight_layout()
plt.show()
```

In [ ]:
```
# Apply PCA to digits
scaler_digits = StandardScaler()
X_digits_scaled = scaler_digits.fit_transform(X_digits)

# Reduce from 64D to 2D for visualization
pca_digits = PCA(n_components=2)
X_digits_pca = pca_digits.fit_transform(X_digits_scaled)
```

```
print(f"Reduced from {X_digits.shape[1]}D to {X_digits_pca.shape[1]}D")
print(f"Variance explained: {np.sum(pca_digits.explained_variance_ratio_)*100:.2f}%

# Visualize in 2D
plt.figure(figsize=(12, 10))
scatter = plt.scatter(X_digits_pca[:, 0], X_digits_pca[:, 1],
                      c=y_digits, cmap='tab10', alpha=0.7, s=30, edgecolors='k', li
plt.colorbar(scatter, label='Digit', ticks=range(10))
plt.xlabel(f'PC1 ({pca_digits.explained_variance_ratio_[0]*100:.1f}% variance)', fo
plt.ylabel(f'PC2 ({pca_digits.explained_variance_ratio_[1]*100:.1f}% variance)', fo
plt.title('Handwritten Digits: 64D → 2D using PCA', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\nObservations:")
print("- Some digits form distinct clusters (e.g., 0, 1, 6)")
print("- Other digits overlap (e.g., 3, 5, 8)")
print("- PCA captures enough structure to separate many classes")
```

In [ ]:

```
# Reconstruction: How much information is lost?
# Compare different numbers of components
n_components_list = [2, 5, 10, 20, 30, 64]

fig, axes = plt.subplots(2, len(n_components_list), figsize=(18, 6))

# Select one digit to reconstruct
sample_idx = 0
original_image = X_digits[sample_idx].reshape(8, 8)

for i, n_comp in enumerate(n_components_list):
    # Apply PCA with n_comp components
    pca_temp = PCA(n_components=n_comp)
    X_temp = pca_temp.fit_transform(X_digits_scaled)
    X_reconstructed = pca_temp.inverse_transform(X_temp)

    # Inverse standardization
    X_reconstructed = X_reconstructed * scaler_digits.scale_ + scaler_digits.mean_
    reconstructed_image = X_reconstructed[sample_idx].reshape(8, 8)

    # Calculate reconstruction error
    mse = np.mean((original_image - reconstructed_image) ** 2)
    var_explained = np.sum(pca_temp.explained_variance_ratio_) * 100

    # Original
    axes[0, i].imshow(original_image, cmap='gray')
    axes[0, i].set_title('Original', fontsize=10)
    axes[0, i].axis('off')

    # Reconstructed
    axes[1, i].imshow(reconstructed_image, cmap='gray')
    axes[1, i].set_title(f'{n_comp} PCs\n{var_explained:.1f}% var\nMSE: {mse:.2f}',
    axes[1, i].axis('off')

plt.suptitle('Image Reconstruction with Different Numbers of Principal Components',
             fontsize=14, fontweight='bold')
```

```
plt.tight_layout()
plt.show()

print("\nKey Insights:")
print("- With only 2 components: Digit is barely recognizable")
print("- With 10-20 components: Digit is clearly recognizable")
print("- With 64 components: Perfect reconstruction (no compression)")
print("\nTrade-off: Compression vs. Information Loss")
```

Part 9: Limitations and Considerations

Limitations of PCA

1. **Linearity**: PCA only captures linear relationships
   - Solution: Use kernel PCA or other nonlinear methods (t-SNE, UMAP)
2. **Variance ≠ Information**: High variance doesn't always mean high importance
   - Example: Outliers can create high-variance components that are just noise
3. **Interpretability**: Principal components are linear combinations of all features
   - Hard to interpret what each PC "means"
4. **Scaling Sensitivity**: Results depend heavily on feature scaling
   - Always standardize before PCA
5. **Assumes Gaussian Distribution**: Works best when data is approximately normal

When NOT to Use PCA

- When interpretability of original features is critical
- When relationships are highly nonlinear
- When you have very few features already
- When features have very different meanings (e.g., mixing categorical and continuous)

Best Practices

1. **Always standardize** your data before PCA
2. **Check explained variance** to choose appropriate k
3. **Visualize** the results (scree plot, biplots)
4. **Consider alternatives** for nonlinear data (kernel PCA, t-SNE, UMAP)
5. **Validate** on downstream tasks (classification, clustering, etc.)

In [ ]:
```
# Demonstration: Effect of scaling
# Create data with features on different scales
np.random.seed(42)
X_unscaled = np.random.randn(200, 2)
```

```
X_unscaled[:, 0] = X_unscaled[:, 0] * 100  # Feature 1: large scale
X_unscaled[:, 1] = X_unscaled[:, 1] * 1    # Feature 2: small scale

# PCA without scaling
pca_unscaled = PCA(n_components=2)
X_pca_unscaled = pca_unscaled.fit_transform(X_unscaled)

# PCA with scaling
scaler_demo = StandardScaler()
X_scaled = scaler_demo.fit_transform(X_unscaled)
pca_scaled = PCA(n_components=2)
X_pca_scaled = pca_scaled.fit_transform(X_scaled)

# Visualize
fig, axes = plt.subplots(2, 2, figsize=(14, 12))

# Original unscaled data
axes[0, 0].scatter(X_unscaled[:, 0], X_unscaled[:, 1], alpha=0.6, s=30)
axes[0, 0].set_xlabel('Feature 1 (scale ~100)', fontsize=11)
axes[0, 0].set_ylabel('Feature 2 (scale ~1)', fontsize=11)
axes[0, 0].set_title('Original Data (Unscaled)', fontsize=12, fontweight='bold')
axes[0, 0].grid(True, alpha=0.3)

# PCA without scaling
axes[0, 1].scatter(X_pca_unscaled[:, 0], X_pca_unscaled[:, 1], alpha=0.6, s=30, col
axes[0, 1].set_xlabel('PC1', fontsize=11)
axes[0, 1].set_ylabel('PC2', fontsize=11)
axes[0, 1].set_title(f'PCA WITHOUT Scaling\nPC1: {pca_unscaled.explained_variance_r
                     fontsize=12, fontweight='bold')
axes[0, 1].grid(True, alpha=0.3)

# Original scaled data
axes[1, 0].scatter(X_scaled[:, 0], X_scaled[:, 1], alpha=0.6, s=30, color='green')
axes[1, 0].set_xlabel('Feature 1 (standardized)', fontsize=11)
axes[1, 0].set_ylabel('Feature 2 (standardized)', fontsize=11)
axes[1, 0].set_title('Scaled Data (Standardized)', fontsize=12, fontweight='bold')
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].axis('equal')

# PCA with scaling
axes[1, 1].scatter(X_pca_scaled[:, 0], X_pca_scaled[:, 1], alpha=0.6, s=30, color='
axes[1, 1].set_xlabel('PC1', fontsize=11)
axes[1, 1].set_ylabel('PC2', fontsize=11)
axes[1, 1].set_title(f'PCA WITH Scaling\nPC1: {pca_scaled.explained_variance_ratio_
                     fontsize=12, fontweight='bold')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nEffect of Scaling on PCA:")
print("=" * 60)
print("\nWITHOUT Scaling:")
print(f"  PC1 explains: {pca_unscaled.explained_variance_ratio_[0]*100:.2f}%")
print(f"  PC2 explains: {pca_unscaled.explained_variance_ratio_[1]*100:.2f}%")
print("  → PC1 dominated by high-variance Feature 1")
```

```
print("\nWITH Scaling:")
print(f"  PC1 explains: {pca_scaled.explained_variance_ratio_[0]*100:.2f}%")
print(f"  PC2 explains: {pca_scaled.explained_variance_ratio_[1]*100:.2f}%")
print("  → Both features contribute more equally")

print("\n" + "=" * 60)
print("⚠  Always standardize your data before PCA!")
```

Part 10: Practice Exercises

Exercise 1: Wine Dataset

Apply PCA to the wine dataset and visualize the results.

In [ ]:
```
# Load wine dataset
from sklearn.datasets import load_wine

wine = load_wine()
X_wine = wine.data
y_wine = wine.target

print("Wine Dataset")
print("=" * 60)
print(f"Number of samples: {X_wine.shape[0]}")
print(f"Number of features: {X_wine.shape[1]}")
print(f"Feature names: {wine.feature_names}")
print(f"Target names: {wine.target_names}")

# TODO: Your task
# 1. Standardize the data
scaler = StandardScaler()
X_wine_scaled = scaler.fit_transform(X_wine)
# 2. Apply PCA to reduce to 2 components
pca = PCA(n_components=2)
X_wine_pca = pca.fit_transform(X_wine_scaled)
# 3. Visualize the results colored by wine type
component_df = pd.DataFrame(
    pca.components_,
    columns=wine.feature_names,
    index=["PC1", "PC2"]
)
print(component_df)
colors = ['red', 'green', 'blue']

plt.figure(figsize=(8,4))

for i, (target, name) in enumerate(zip(range(3), wine.target_names)):
    mask = y_wine == target
    plt.scatter(X_wine_pca[mask,0], X_wine_pca[mask,1], c=colors[i], label=name,
                cmap='viridis', s=100, edgecolor='k')
plt.legend()
```

```
plt.show()
# 4. Create a scree plot
pca_scree = PCA()
pca_scree.fit(X_wine_scaled)

plt.figure(figsize=(8,4))

plt.plot(np.arange(0,len(pca_scree.explained_variance_)), pca_scree.explained_varia
         'ro-', alpha=0.6, markersize=8)
plt.grid(True, alpha=0.3)

plt.show()


# 5. Interpret the results

print("\n📝 Your turn! Complete the exercise above.")
```

In [ ]:

```
# Solution to Exercise 1
print("Solution to Exercise 1")
print("=" * 60)

# 1. Standardize
scaler_wine = StandardScaler()
X_wine_scaled = scaler_wine.fit_transform(X_wine)

# 2. Apply PCA
pca_wine = PCA(n_components=2)
X_wine_pca = pca_wine.fit_transform(X_wine_scaled)

print(f"Variance explained by 2 components: {np.sum(pca_wine.explained_variance_rat

# 3. Visualize
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

colors = ['red', 'green', 'blue']
for i, (target, name) in enumerate(zip(range(3), wine.target_names)):
    mask = y_wine == target
    axes[0].scatter(X_wine_pca[mask, 0], X_wine_pca[mask, 1],
                    c=colors[i], label=name, alpha=0.7, s=60, edgecolors='k', linew

axes[0].set_xlabel(f'PC1 ({pca_wine.explained_variance_ratio_[0]*100:.1f}% variance
axes[0].set_ylabel(f'PC2 ({pca_wine.explained_variance_ratio_[1]*100:.1f}% variance
axes[0].set_title('Wine Dataset: PCA Projection', fontsize=14, fontweight='bold')
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)

# 4. Scree plot
pca_wine_full = PCA()
pca_wine_full.fit(X_wine_scaled)

axes[1].bar(range(1, len(pca_wine_full.explained_variance_) + 1),
            pca_wine_full.explained_variance_, alpha=0.7, color='steelblue')
axes[1].plot(range(1, len(pca_wine_full.explained_variance_) + 1),
             pca_wine_full.explained_variance_, 'ro-', linewidth=2, markersize=8)
axes[1].set_xlabel('Principal Component', fontsize=12)
```

```
axes[1].set_ylabel('Eigenvalue', fontsize=12)
axes[1].set_title('Scree Plot', fontsize=14, fontweight='bold')
axes[1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# 5. Interpretation
print("\nInterpretation:")
print("- The three wine classes are well-separated in the 2D PCA space")
print("- First two components capture most of the variance")
print("- PCA successfully reduces 13D to 2D while preserving class structure")
```

Exercise 2: Create Your Own Dataset

Create a synthetic 3D dataset and apply PCA to reduce it to 2D.

In [ ]:

```
# TODO: Your task
np.random.seed(42)
# 1. Create a 3D dataset with correlation between features
data_3d_covariance = [[2,3,1],[3,1,3],[1,2,3]]
Q = np.cov(data_3d_covariance)
data = np.random.multivariate_normal([0,0,0], Q, size=1000)
# print(data)

# 2. Visualize in 3D
# from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(10,8))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(data[:,0], data[:,1], data[:,2], c=data[:,2],
    cmap='viridis', alpha=0.6, edgecolors='w', s=20)
fig.colorbar(scatter, label="Z value", ax=ax)
ax.view_init(elev=20, azim=45)
plt.tight_layout()
plt.show()

# 3. Apply PCA to reduce to 2D
scaler = StandardScaler()
X_scaled = scaler.fit_transform(data)

pca = PCA(n_components=2)
X_pca_scaled = pca.fit_transform(X_scaled)
# 4. Visualize in 2D
plt.figure(figsize=(10,8))

plt.scatter(X_pca_scaled[:,0], X_pca_scaled[:,1], cmap='viridis', alpha=0.6, s=30,
plt.show()

# 5. Compare information loss
pca = PCA()
pca.fit(data)
print(pca.explained_variance_ratio_)
csum = np.cumsum(pca.explained_variance_ratio_)
print(f"{csum[1]:.2f}")
```

```
print("📝 Your turn! Create and analyze your own dataset.")
```

Summary

Key Takeaways

1. **PCA is a linear dimensionality reduction technique** that finds directions of maximum variance

2. **Mathematical foundation**:

   - Based on eigendecomposition of covariance matrix
   - Eigenvectors = principal components (directions)
   - Eigenvalues = variance explained by each component
3. **Algorithm steps**:

   - Standardize data
   - Compute covariance matrix
   - Find eigenvalues and eigenvectors
   - Project data onto top k eigenvectors
4. **Choosing k components**:

   - Use scree plot to find elbow
   - Keep components until reaching variance threshold (e.g., 95%)
   - Consider downstream task performance
5. **Applications**:

   - Visualization (2D/3D)
   - Noise reduction
   - Feature extraction
   - Data compression
   - Preprocessing for ML
6. **Limitations**:

   - Only captures linear relationships
   - Sensitive to scaling
   - Loss of interpretability
   - Assumes variance = importance
7. **Best practices**:

   - Always standardize first
   - Visualize explained variance
   - Consider alternatives for nonlinear data
   - Validate on downstream tasks

Further Reading

- **Kernel PCA**: Nonlinear extension using kernel trick
- **t-SNE**: Nonlinear dimensionality reduction for visualization
- **UMAP**: Modern alternative to t-SNE
- **Autoencoders**: Neural network-based dimensionality reduction
- **Factor Analysis**: Probabilistic alternative to PCA

Practice Recommendations

1. Apply PCA to real datasets (Kaggle, UCI ML Repository)
2. Compare PCA with other dimensionality reduction methods
3. Use PCA as preprocessing for classification/regression
4. Experiment with different numbers of components
5. Visualize high-dimensional data using PCA

End of Tutorial

Prepared for DA5400W - Foundations of Machine Learning
Dr. Arun B Ayyar, IIT Madras